Práctica 2: Ejercicios de Unity básicos

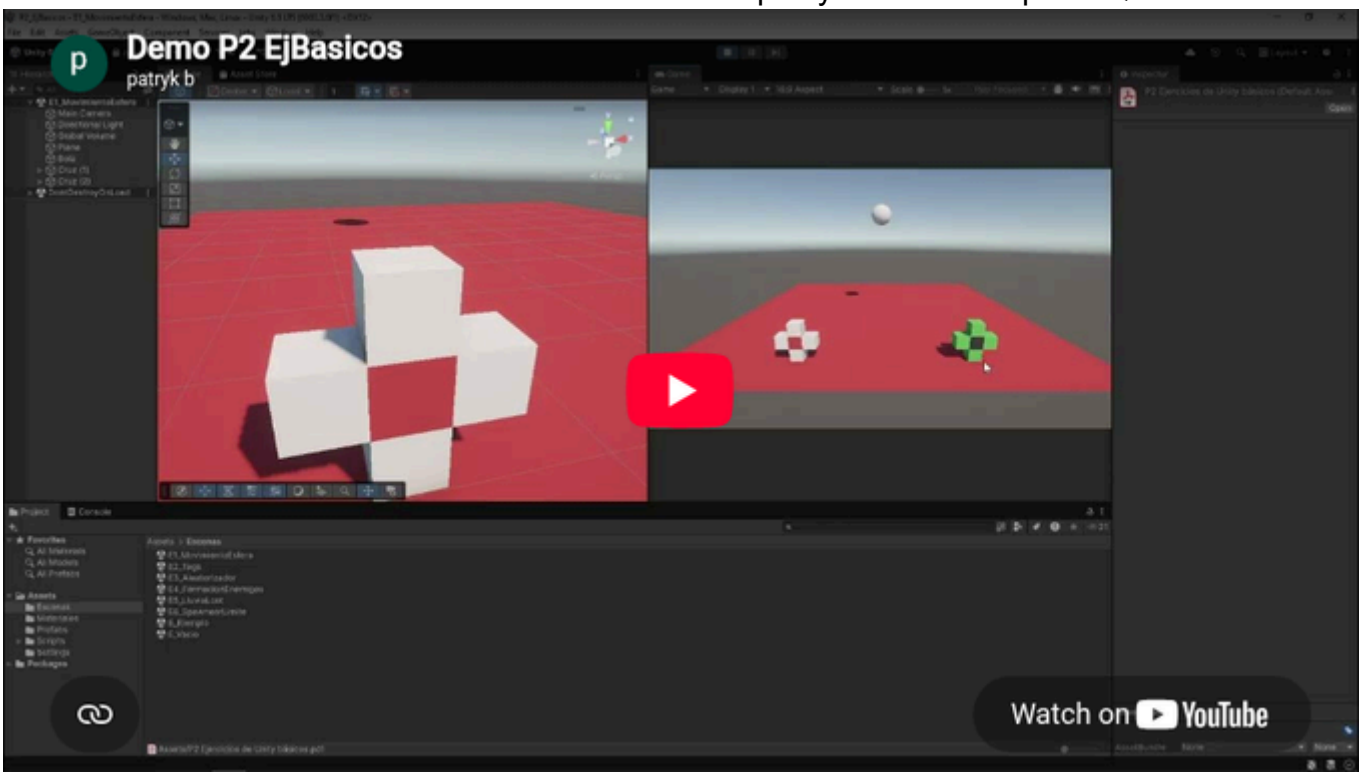
Índice:

- Preparación
- E1_MovimientoEsfera
- E2_Tags
- E3_Aleatorizador
- E4_FormacionEnemigos
- E5_LluviaLoot
- E6_SpawnearLimite
- Conclusión

Para más información sobre el proyecto de Unity:

https://github.com/ptrk02/Clases26-27/tree/main/Unity/P2_EjBasicos

Enlace a video demostrativo en YT: https://youtu.be/TakDpcUIvQE

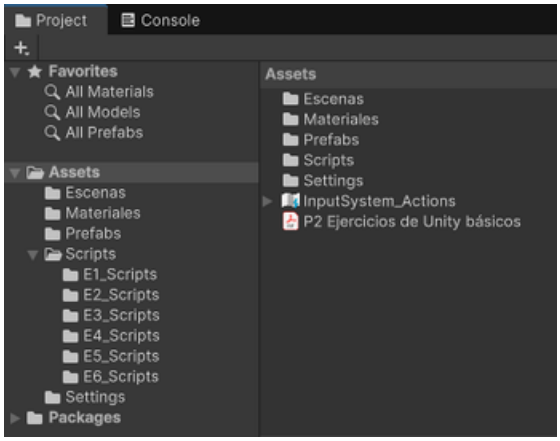


Práctica 2: Ejercicios de Unity básicos

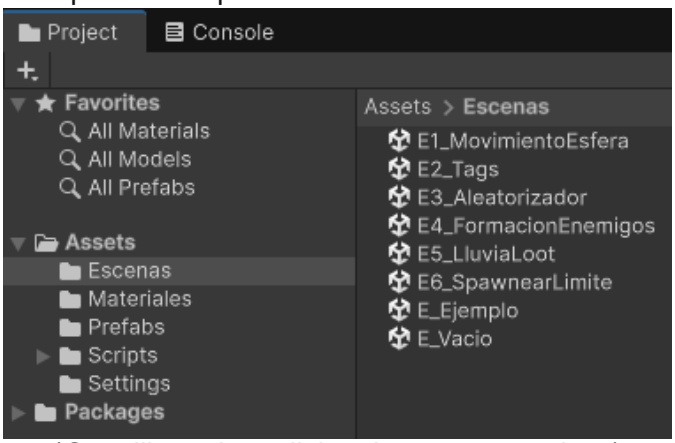
Preparación:

En este apartado crearemos los assets y configuraremos el proyecto de Unity

Creamos las carpetas necesarias:

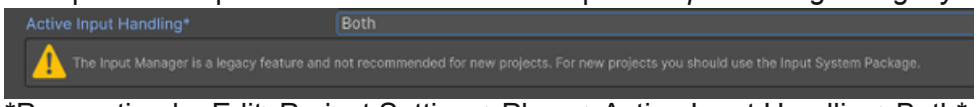


Esta práctica requiere de 6 escenas distintas:

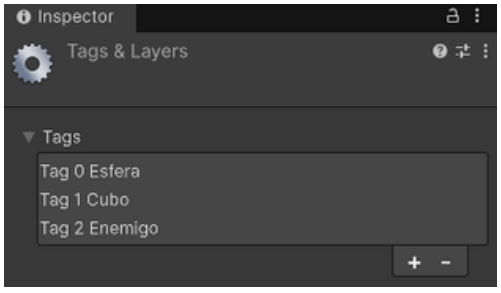


Se utilizan dos adicionales para prototipar

Esta práctica depende de funciones usadas por el *Input Manager Legacy*.



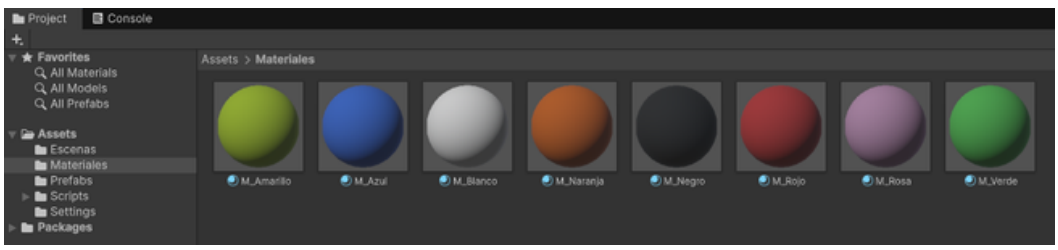
Para activarlo: Edit>Project Settings>Player>Active Input Handling>Both



En esta práctica daremos uso a las Tags. Hay que añadir estas tres en nuestro proyecto de Unity:

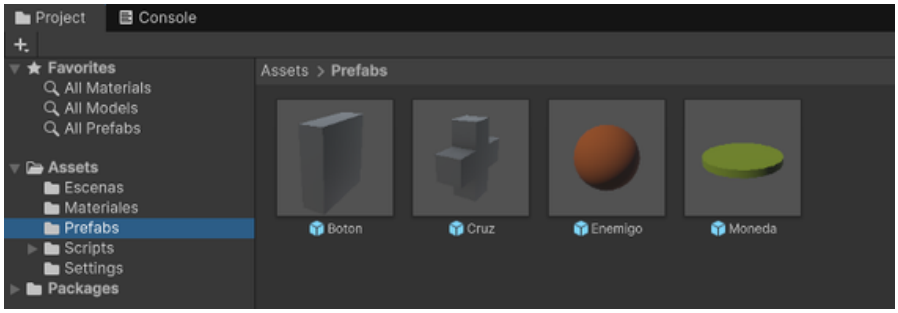
- Esfera
- Cubo
- Enemigo

Nuestros elementos de la escena deben distinguirse, para ello creamos los siguientes materiales;



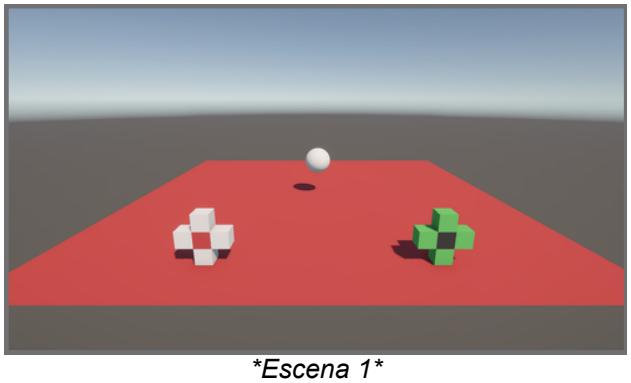
Práctica 2: Ejercicios de Unity básicos

Las composiciones de las escenas están formadas por formas básicas como cubos y esferas. Además se usan prefabs para ayudar a la implementación mediante scripting. Esta práctica usa estos prefabs:

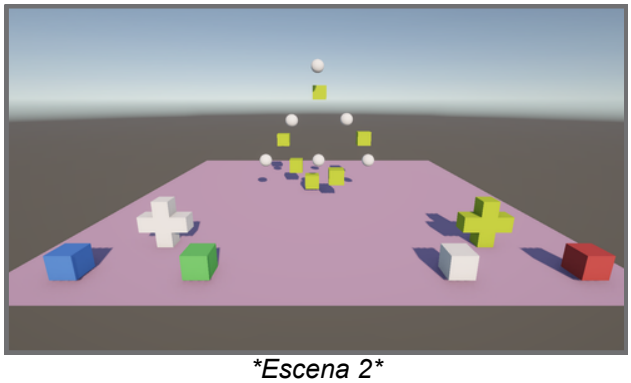


Los dos últimos prefabs son estrictamente necesarios para que la práctica funcione correctamente

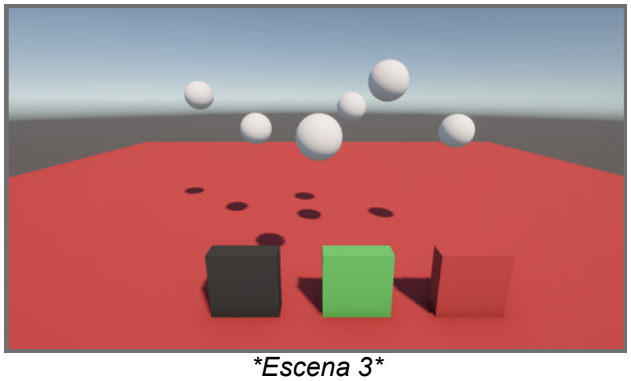
Por último haremos el ensamblaje final de las escenas tomando como referencia la documentación proporcionada: P2 Ejercicios de Unity básicos.pdf



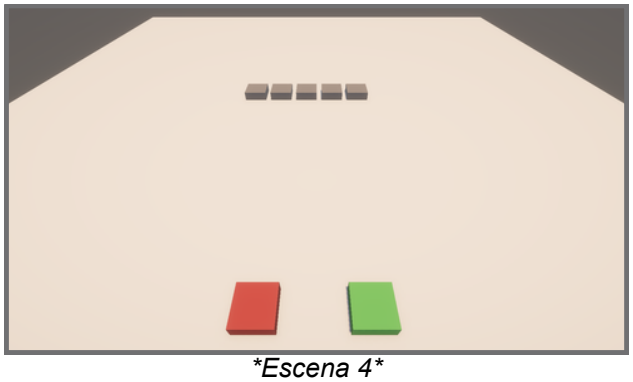
Escena 1



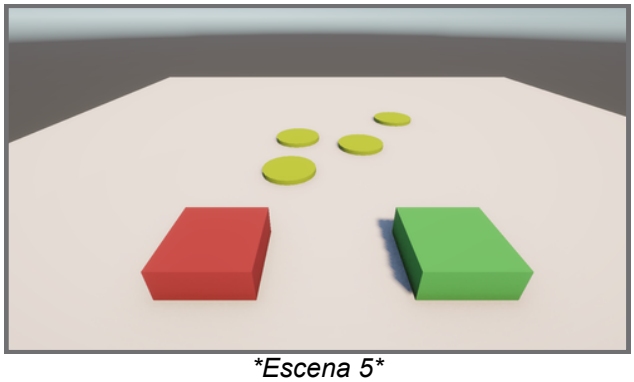
Escena 2



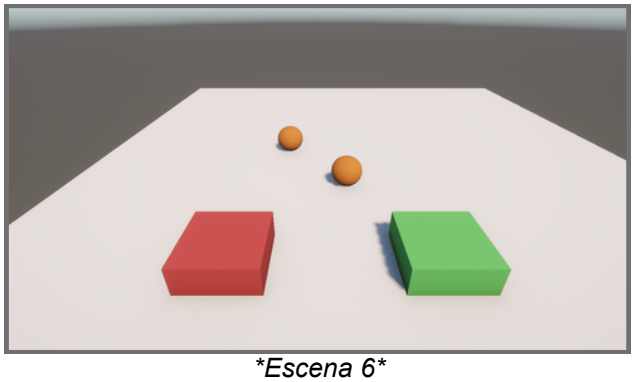
Escena 3



Escena 4

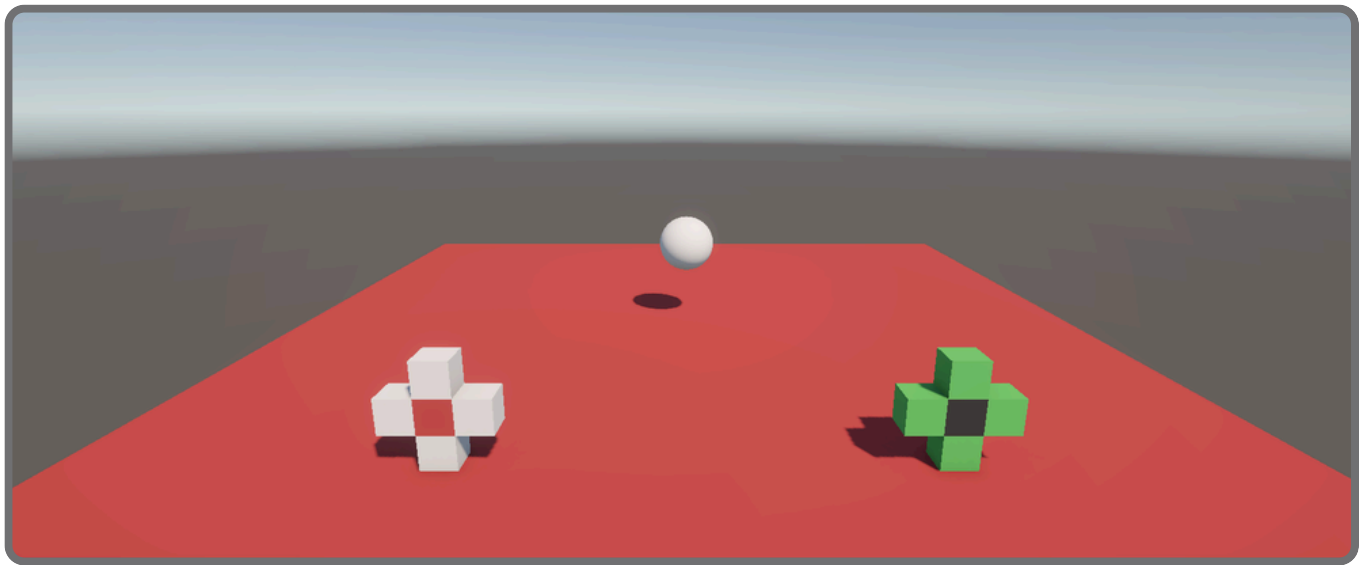


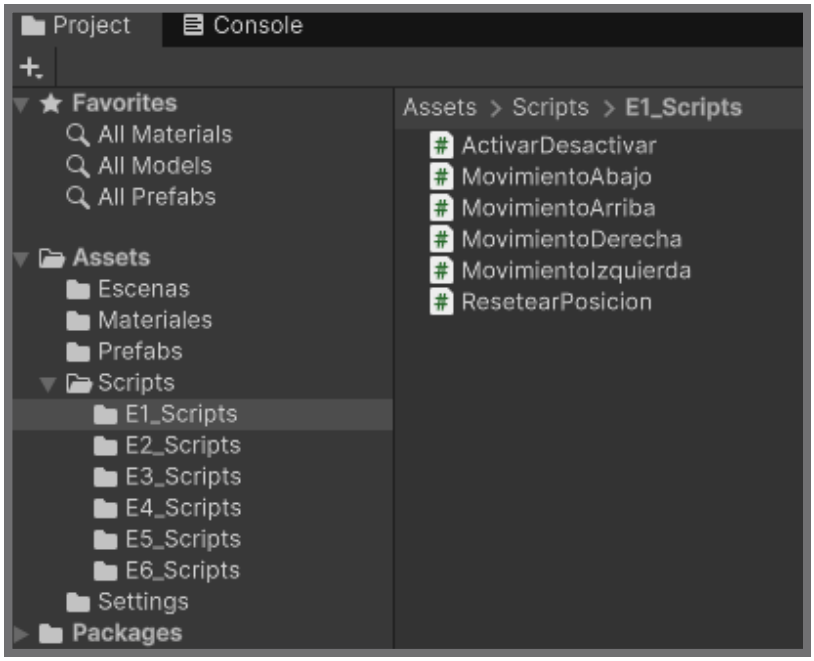
Escena 5



Escena 6

E1_MovimientoEsfera:





Para la primera escena se crean **seis scripts** de MonoBehaviour con las siguientes directrices:

- La cruceta blanca mueve la esfera 1 unidad en la dirección del botón (haz solo arriba e izquierda)
- El cuadrado rojo resetea la posición.
- La cruceta verde mueve la esfera 3 unidades en la dirección del botón (haz solo abajo y derecha).
- El cuadrado negro lo activa y desactiva.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

MovimientoArriba.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// La cruceta blanca mueve la esfera 1 unidad en la dirección del botón
// (haz solo arriba e izquierda)
//
//
using UnityEngine;

public class MovimientoArriba: MonoBehaviour
{
    GameObject bola;

    void Start()
    {
        // Busca por nombre GameObject en Hierarchy
        bola = GameObject.Find("Bola");
    }

    void OnMouseDown()
    {
        // Creamos variables locales de position
        float posX = bola.transform.position.x;
        float posY = bola.transform.position.y;
        float posZ = bola.transform.position.z;

        if (posY - 1 < 1.5f) // Limite de posicion en Y
        {
            posY = 1.5f;
        }

        // Asignamos los componentes de position a bola
        bola.transform.position = new Vector3(posX, posY +1, posZ);
    }
}
```

MovimientoArriba está asignado a la cruceta blanca superior.

MovimientoIzquierda.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// La cruceta blanca mueve la esfera 1 unidad en la dirección del botón
// (haz solo arriba e izquierda)
//
//
using UnityEngine;

public class MovimientoIzquierda: MonoBehaviour
{
    GameObject bola;

    void Start()
    {
        bola = GameObject.Find("Bola"); // Busca por nombre GameObject en Hierarchy
    }

    void OnMouseDown()
    {
        // Creamos variables locales de position
        float posX = bola.transform.position.x;
        float posY = bola.transform.position.y;
        float posZ = bola.transform.position.z;

        if (posX - 1 <= -4f) // Limite de posicion en Y
        {
            posX = -4f;
        }

        bola.transform.position = new Vector3(posX -1, posY, posZ);
    }
}
```

MovimientoIzquierda está asignado a la cruceta blanca izquierda.

ResetearPosicion.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// El cuadrado rojo resetea la posición.
//
//
using UnityEngine;

public class ResetearPosicion: MonoBehaviour
{
    GameObject bola;

    void Start()
    {
        // Busca por nombre GameObject en Hierarchy
        bola = GameObject.Find("Bola");
    }

    void OnMouseDown()
    {
        // Asignamos nuevo vector a posicion actual
        bola.transform.position = new Vector3(0, 1, 0);
    }
}
```

ResetearPosicion está asignado al cuadrado rojo.

MovimientoAbajo.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// La cruceta verde mueve la esfera 3 unidades en la dirección del botón
// (haz solo abajo y derecha).
//
//
using UnityEngine;

public class MovimientoAbajo: MonoBehaviour
{
    GameObject bola;

    void Start()
    {
        // Busca por nombre GameObject en Hierarchy
        bola = GameObject.Find("Bola");
    }

    void OnMouseDown()
    {
        // Creamos variables locales de position
        float posX = bola.transform.position.x;
        float posY = bola.transform.position.y;
        float posZ = bola.transform.position.z;

        if (posY - 1 < +0.5f) // Limite de posicion en Y
        {
            posY = +3.5f;
        }

        bola.transform.position = new Vector3(posX, posY -3, posZ);
    }
}
```

MovimientoAbajo está asignado a la cruceta verde inferior.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

MovimientoDerecha.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// La cruceta verde mueve la esfera 3 unidades en la
// dirección del botón (haz solo abajo y derecha).
//
//
using UnityEngine;

public class MovimientoDerecha: MonoBehaviour
{
    GameObject bola;

    void Start()
    {
        // Busca por nombre GameObject en Hierarchy
        bola = GameObject.Find("Bola");
    }

    void OnMouseDown()
    {
        // Creamos variables locales de position
        float posX = bola.transform.position.x;
        float posY = bola.transform.position.y;
        float posZ = bola.transform.position.z;

        if (posX + 1 >= +4f) // Limite de posicion en X
        {
            posX = +4f;
        }

        bola.transform.position = new Vector3(posX +3, posY, posZ);
    }
}
```

MovimientoDerecha está asignado a la cruceta verde derecha.

ActivarDesactivar.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 09/11/2026
// El cuadrado negro lo activa y desactiva.
//
//
using UnityEngine;

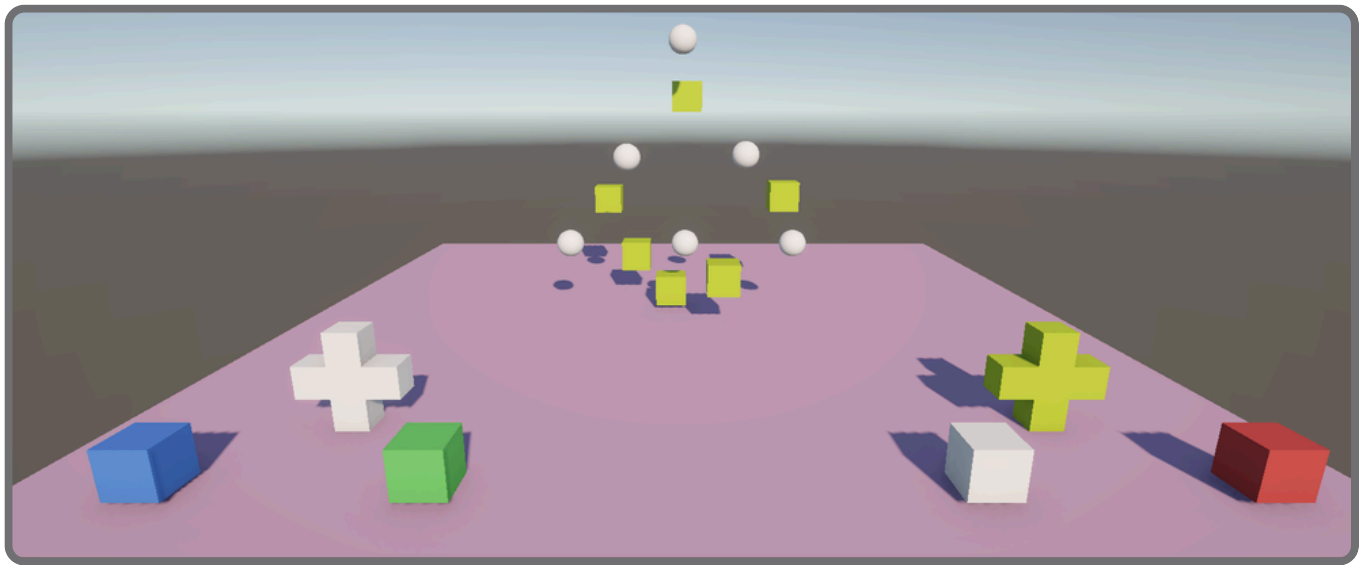
public class ActivarDesactivar: MonoBehaviour
{
    GameObject bola;

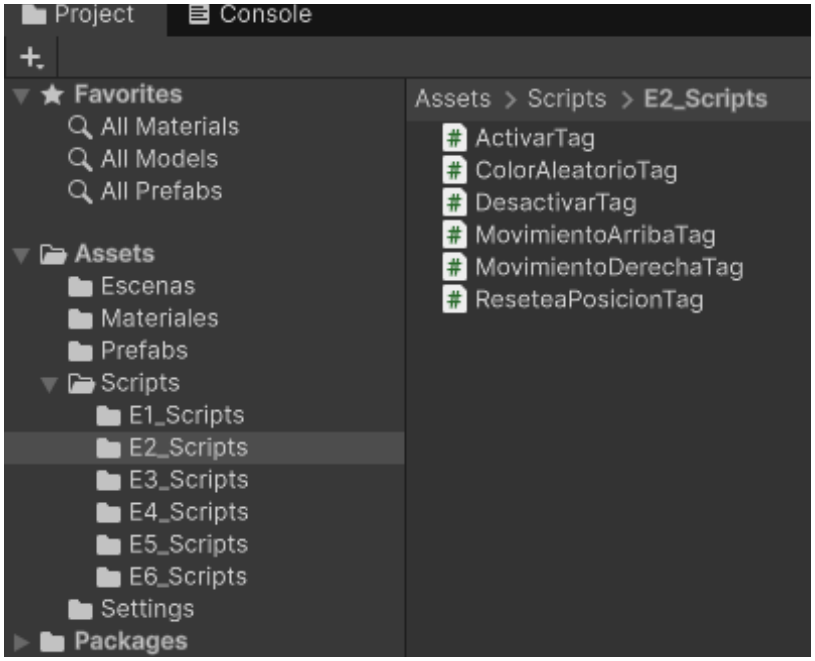
    void Start()
    {
        bola = GameObject.Find("Bola");
    }

    void OnMouseDown()
    {
        // Asignamos la inversa del parámetro que devuelve el metodo activeSelf de bola.
        // activeSelf es un método que comprueba si un objeto está activo en la escena.
        bola.SetActive(!bola.activeSelf);
    }
}
```

ActivarDesactivar está asignado al cuadrado negro.

E2_Tags:





Para la segunda escena se crean **seis scripts** de MonoBehaviour con las siguientes directrices:

- La cruceta de la derecha mueve todas las esferas con TAG esfera 1 unidad en la dirección del botón (haz solo arriba).
- La cruceta de la izquierda mueve todos los cubos con TAG cubo 1 unidad en la dirección del botón (haz solo derecha).
- Un cuadrado rojo resetea todos los cubos y esferas a su posición inicial.
- Un botón azul desactiva las esferas.
- Un botón verde activa las esferas.
- Un botón blanco pinta cada esfera de un color aleatorio.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

MovimientoArribaTag.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// La cruceta de la derecha mueve todas las esferas con TAG
// esfera 1 unidad en la dirección del botón (haz solo arriba).
//
//
using UnityEngine;

public class MovimientoArribaTag: MonoBehaviour
{
    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Tag "Esfera"
        foreach (GameObject bola in GameObject.FindGameObjectsWithTag("Esfera"))
        {
            // Declaro nuevo vector
            Vector3 posicion = bola.transform.position;

            // Modifico el nuevo vector
            posicion.y += 1f;

            // Aplico vector modificado a transform actual
            bola.transform.position = posicion;
        }
    }
}
```

MovimientoArribaTag está asignado a la cruceta verde superior.

MovimientoDerechaTag.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// La cruceta de la izquierda mueve todos los cubos con TAG
// cubo 1 unidad en la dirección del botón (haz solo derecha).
//
//
using UnityEngine;

public class MovimientoDerechaTag: MonoBehaviour
{
    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Tag "Cubo"
        foreach (GameObject bola in GameObject.FindGameObjectsWithTag("Cubo"))
        {
            // Declaro nuevo vector
            Vector3 posicion = bola.transform.position;

            // Modifico el nuevo vector
            posicion.x += 1f;

            // Aplico vector modificado a transform actual
            bola.transform.position = posicion;
        }
    }
}
```

MovimientoDerecha está asignado a la cruceta blanca derecha.

ReseteaPosicionTag.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// Un cuadrado rojo resetea todos los cubos y esferas a su posición inicial.
// PD: Recorremos varios tags duplicando código, habrían mejores maneras de hacerlo.
//

using UnityEngine;

public class ReseteaPosicionTag : MonoBehaviour
{
    GameObject[] bolas;
    GameObject[] cubos;
    Vector3[] posicionesInicialesEsfera;
    Vector3[] posicionesInicialesCubo;

    void Start()
    {
        // Buscar todos los objetos con el Tag
        bolas = GameObject.FindGameObjectsWithTag("Esfera");
        cubos = GameObject.FindGameObjectsWithTag("Cubo");

        // Crear un array para guardar sus posiciones iniciales
        posicionesInicialesEsfera = new Vector3[bolas.Length];
        posicionesInicialesCubo = new Vector3[cubos.Length];

        // Guardo la posición inicial de cada objeto
        for (int i = 0; i < bolas.Length; i++)
        {
            posicionesInicialesEsfera[i] = bolas[i].transform.position;
        }
        for (int i = 0; i < cubos.Length; i++)
        {
            posicionesInicialesCubo[i] = cubos[i].transform.position;
        }
    }

    void OnMouseDown()
    {
        // Restauro las posiciones iniciales de los gameObjects de las dos Tags
        for (int i = 0; i < bolas.Length; i++)
        {
            bolas[i].transform.position = posicionesInicialesEsfera[i];
        }
        for (int i = 0; i < cubos.Length; i++)
        {
            cubos[i].transform.position = posicionesInicialesCubo[i];
        }
    }
}
```

ResetearPosicionTag está asignado al botón rojo.

DesactivarTag.cs

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// Un botón azul desactiva las esferas.
//
//
using UnityEngine;

public class DesactivarTag: MonoBehaviour
{
    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Tag "Esfera"
        foreach (GameObject bola in GameObject.FindGameObjectsWithTag("Esfera"))
        {
            // Los asigno como inactivos en la escena
            bola.SetActive(false);
        }
    }
}
```

DesactivarTag está asignado al botón azul.

A continuación, se muestra el código de cada script:

ActivarTag.cs

Assets > Scripts > E2_Scripts > ActivarTag.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// Un botón verde activa las esferas.
//
//
using UnityEngine;

public class ActivarTag: MonoBehaviour
{
    GameObject[] bolas;

    void Start()
    {
        // Ponemos los GameObjects con la misma tag en una array
        bolas = GameObject.FindGameObjectsWithTag("Esfera");
    }

    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Tag "Esfera"
        foreach (GameObject bola in bolas)
        {
            // Los asigno como activos en la escena
            bola.SetActive(true);
        }
    }
}
```

ActivarTag está asignado al botón verde.

ColorAleatorioTag.cs

Assets > Scripts > E2_Scripts > ColorAleatorioTag.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// Un botón blanco pinta cada esfera de un color aleatorio.
//
//
using UnityEngine;

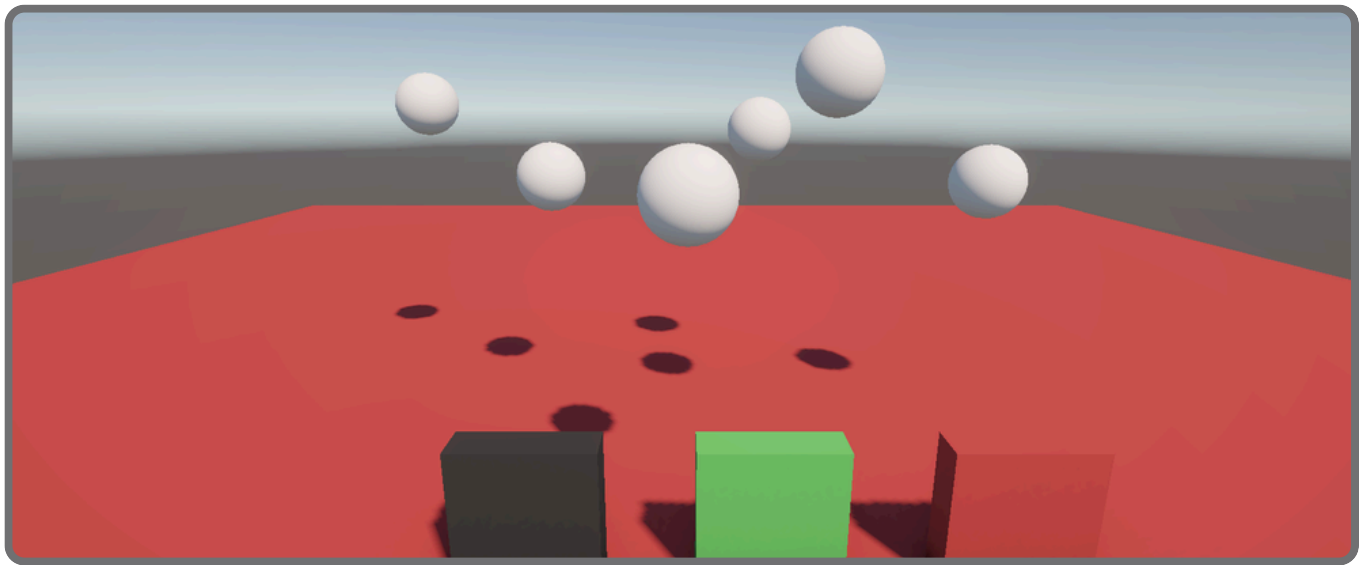
public class ColorAleatorioTag: MonoBehaviour
{
    GameObject[] bolas;

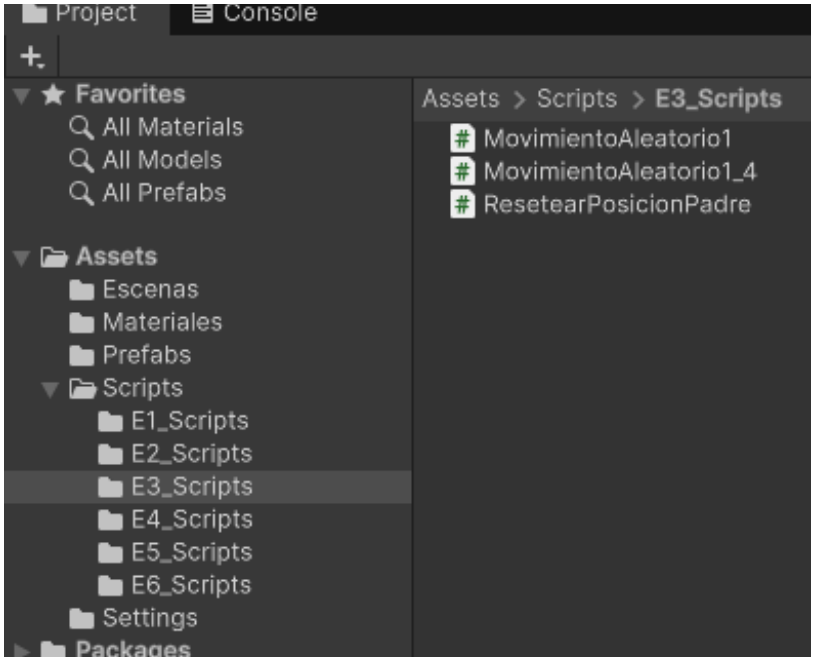
    void Start()
    {
        // Ponemos los GameObjects con la misma tag en una array
        bolas = GameObject.FindGameObjectsWithTag("Esfera");
    }

    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Tag "Esfera"
        foreach (GameObject bola in bolas)
        {
            // Suponiendo que cada GameObject de la array tiene un Renderer
            bola.GetComponent<Renderer>().material.color = Random.ColorHSV();
        }
    }
}
```

ColorAleatorioTag está asignado al botón blanco.

E3_Aleatorizador:





Para la tercera escena se crean **tres scripts** de MonoBehaviour con las siguientes directrices:

- El primero mueve en alguna dirección aleatoria a cada uno de los hijos. El movimiento es de 1 unidad.
- El segundo mueve X unidades en alguna dirección aleatoria a cada uno de los hijos. La X es una valor aleatorio entre 1 y 4.
- El cuarto (rojo) los resetea a la posición inicial.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

MovimientoAleatorio1.cs

Assets > Scripts > E3_Scripts > MovimientoAleatorio1.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// El primero mueve en alguna dirección aleatoria a cada uno de los hijos.
// El movimiento es de 1 unidad.
//
//
using UnityEngine;

public class MovimientoAleatorio1 : MonoBehaviour
{
    GameObject contenedor;

    void Start()
    {
        contenedor = GameObject.Find("PadreVacio");
    }

    void OnMouseDown()
    {
        // Recorre todos los hijos directos del padre
        foreach (Transform hijo in contenedor.transform)
        {
            // Genera una dirección aleatoria en 3D con longitud exactamente 1
            Vector3 direccion = Random.onUnitSphere;

            // Desplaza cada objeto una unidad desde su posición actual
            hijo.position += direccion;
        }
    }
}
```

MovimientoAleatorio1 está asignado al botón negro.

MovimientoAleatorio1_4.cs

Assets > Scripts > E3_Scripts > MovimientoAleatorio1_4.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// El segundo mueve X unidades en alguna dirección aleatoria a cada uno
// de los hijos. La X es una valor aleatorio entre 1 y 4.
//
//
using UnityEngine;

public class MovimientoAleatorio1_4 : MonoBehaviour
{
    GameObject contenedor;

    void Start()
    {
        contenedor = GameObject.Find("PadreVacio");
    }

    void OnMouseDown()
    {
        // Recorre todos los hijos directos del padre
        foreach (Transform hijo in contenedor.transform)
        {
            // Genera una dirección aleatoria en 3D con longitud exactamente 1,
            // luego multiplica su valor por número aleatrorio; 1 entre 4.
            Vector3 direccion = Random.onUnitSphere * Random.Range(1f, 5f);

            // desplaza cada objeto una unidad desde su posición actual
            hijo.position += direccion;
        }
    }
}
```

MovimientoAleatorio1_4 está asignado al botón verde.

ResetearPosicionPadre.cs

Assets > Scripts > E3_Scripts > ResetearPosicionPadre.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 10/11/2026
// El cuarto (rojo) los resetea a la posición inicial.
//
//
using UnityEngine;

public class ResetearPosicionPadre: MonoBehaviour
{
    GameObject contenedor;
    Transform[] hijos;
    Vector3[] posicionesIniciales;

    void Start()
    {
        contenedor = GameObject.Find("PadreVacio");

        // Obtener cantidad de hijos usando childCount
        int cantidadHijos = contenedor.transform.childCount;

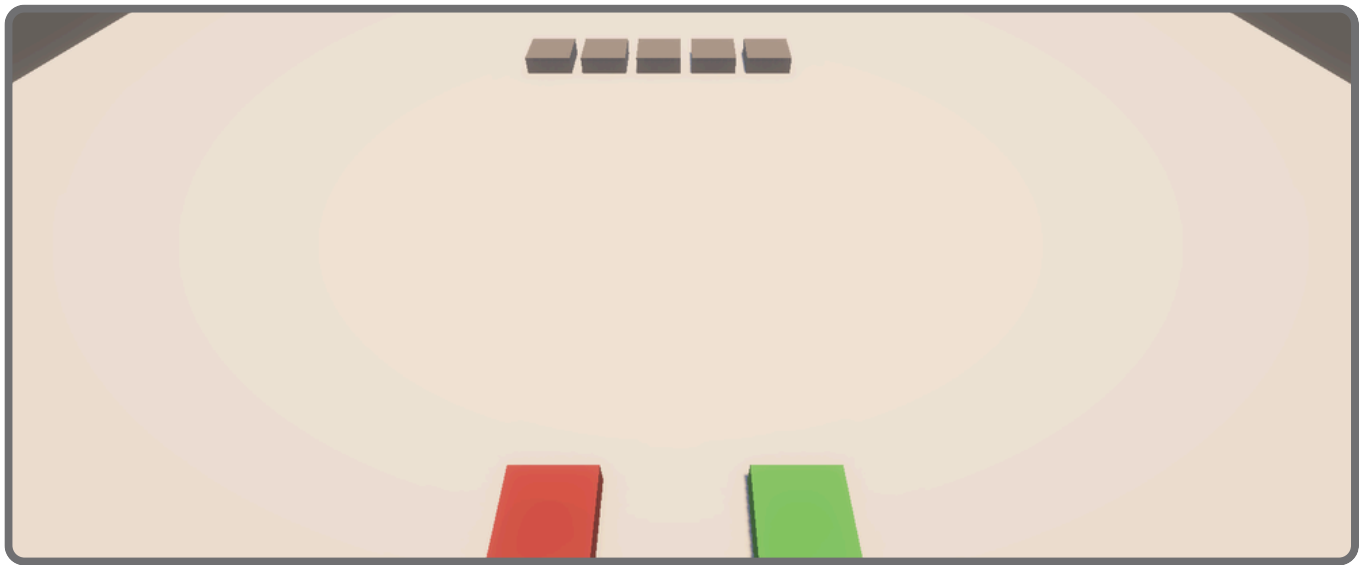
        hijos = new Transform[cantidadHijos];
        posicionesIniciales = new Vector3[cantidadHijos];

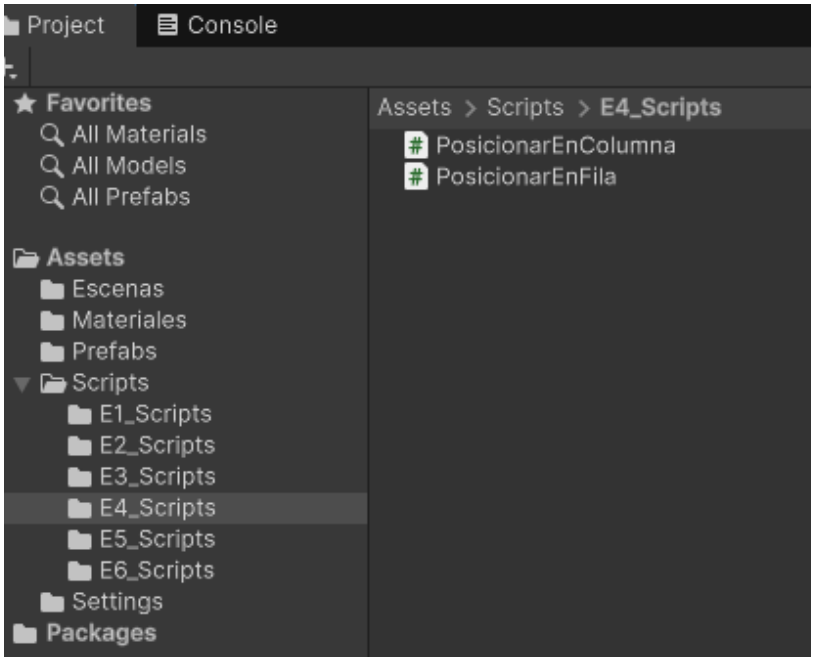
        // Guardar los hijos y sus posiciones iniciales
        for (int i = 0; i < cantidadHijos; i++)
        {
            hijos[i] = contenedor.transform.GetChild(i);
            posicionesIniciales[i] = hijos[i].position;
        }
    }

    void OnMouseDown()
    {
        for (int i = 0; i < hijos.Length; i++)
        {
            // Restaurar la posición de cada hijo
            hijos[i].position = posicionesIniciales[i];
        }
    }
}
```

ResetearPosicionPadre está asignado al botón rojo.

E4_FormacionEnemigos:





Para la cuarta escena se crean **dos scripts** de MonoBehaviour con las siguientes directrices:

- El primero (botón rojo FILA) calcula una posición aleatoria y válida para el primer hijo y coloca el resto de hijos en fila (uno a continuación del otro, con una separación fija entre ellos).
- El segundo (botón verde COLUMNA) hace lo mismo, pero colocando los hijos en columna.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

PosicionarEnColumna.cs M

Assets > Scripts > E4_Scripts > PosicionarEnColumna.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El segundo (botón verde COLUMNA) hace lo mismo,
// pero colocando los hijos en columna.
//
//
using UnityEngine;

0 references | Unity Script (1 asset reference)
public class PosicionarEnColumna : MonoBehaviour
{
    3 references
    GameObject contenedor;

    1 reference
    float minX = -8f;
    1 reference
    float maxX = 8f;
    1 reference
    float minZ = -8f;
    1 reference
    float maxZ = 8f;

    2 references
    float separacion = 1.25f;

    0 references | Unity Message
    void Start()
    {
        contenedor = GameObject.Find("PadreVacio");
    }

    0 references | Unity Message
    void OnMouseDown()
    {
        // Conseguir numero de hijos
        int cantidadHijos = contenedor.transform.childCount;

        // Calcular el espacio total necesario
        float longitudColumna = (cantidadHijos - 1) * separacion;

        // Posición aleatoria para el primer hijo dentro del rango del espacio necesario
        float posX = Random.Range(minX, maxX);
        float posZ = Random.Range(minZ, maxZ - longitudColumna);

        // Colocar cada hijo en fila a partir del primero
        for (int i = 0; i < cantidadHijos; i++)
        {
            Transform hijo = contenedor.transform.GetChild(i);

            Vector3 posicion = hijo.position;

            posicion.x = posX;
            posicion.z = posZ + (i * separacion);

            hijo.position = posicion;
        }
    }
}
```

PosicionEnColumna está asignado al botón verde.

PosicionarEnFila.cs M

Assets > Scripts > E4_Scripts > PosicionarEnFila.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El primero (botón rojo FILA) calcula una posición aleatoria
// y válida para el primer hijo y coloca el resto de hijos en fila
// (uno a continuación del otro, con una separación fija entre ellos).
//
//
using UnityEngine;

0 references | Unity Script (1 asset reference)
public class PosicionarEnFila : MonoBehaviour
{
    3 references
    GameObject contenedor;

    // Area segura
    1 reference
    float minX = -8f;
    1 reference
    float maxX = 8f;
    1 reference
    float minZ = -8f;
    1 reference
    float maxZ = 8f;

    // Puede ser cualquiera, pero esta queda bien
    2 references
    float separacion = 1.25f;

    0 references | Unity Message
    void Start()
    {
        contenedor = GameObject.Find("PadreVacio");
    }

    0 references | Unity Message
    void OnMouseDown()
    {
        // Conseguir numero de hijos
        int cantidadHijos = contenedor.transform.childCount;

        // Calcular el espacio total necesario
        float longitudFila = (cantidadHijos - 1) * separacion;

        // Posición aleatoria para el primer hijo dentro del rango del espacio necesario
        float posX = Random.Range(minX, maxX - longitudFila);
        float posZ = Random.Range(minZ, maxZ);

        for (int i = 0; i < cantidadHijos; i++) // Colocar cada hijo en fila a partir del primero
        {
            Transform hijo = contenedor.transform.GetChild(i);

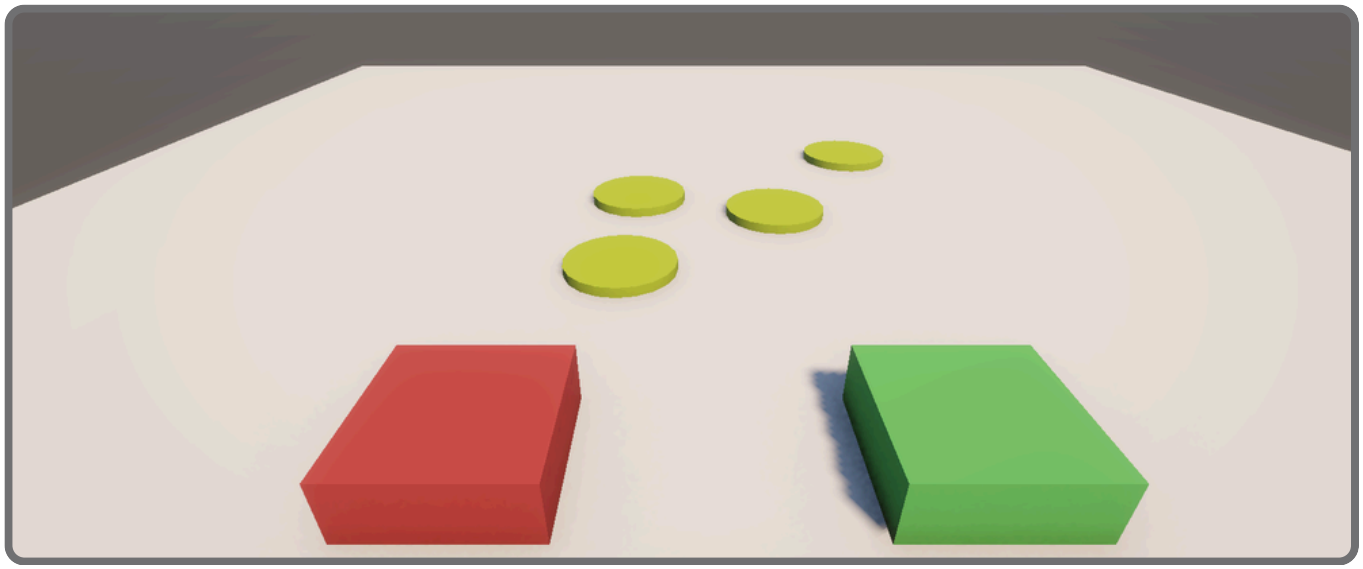
            Vector3 posicion = hijo.position;

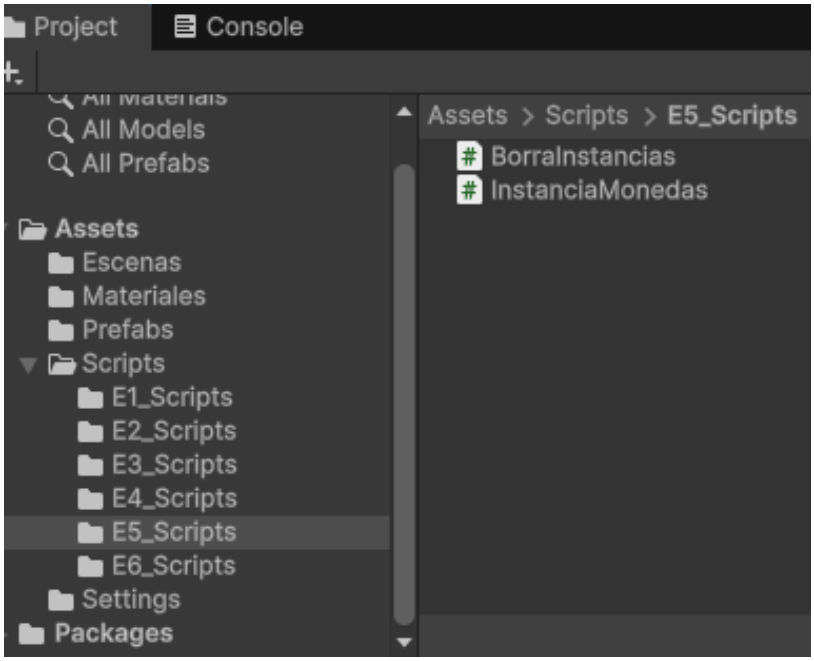
            posicion.x = posX + (i * separacion);
            posicion.z = posZ;

            hijo.position = posicion;
        }
    }
}
```

PosicionEnFila está asignado al botón rojo.

E5_LluviaLoot:





Para la quinta escena se crean **dos scripts** de MonoBehaviour con las siguientes directrices:

- El primero instancia N monedas (a partir del prefab) en posiciones aleatorias dentro del área definida. El valor de N debe poder modificarse desde el Inspector.
- El segundo destruye todas las monedas que haya en la escena en ese momento.

Práctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

InstanciaMonedas.cs M

Assets > Scripts > E5_Scripts > InstanciaMonedas.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El primero instancia N monedas (a partir del prefab) en posiciones aleatorias
// dentro del área definida. El valor de N debe poder modificarse desde el Inspector.
//
//
using UnityEngine;

0 references | Unity Script (1 asset reference)
public class InstanciaMonedas : MonoBehaviour
{
    // [SerializeField] para ser editable desde el inspector
    1 reference | Unity Serialized Field = Moneda
    [SerializeField] GameObject prefabMoneda;

    // N monedas por cada clic, puede ser cualquiera
    1 reference | Unity Serialized Field = 3
    [SerializeField] int cantidadMonedas = 1;

    2 references
    GameObject contenedor;

    // Zona segura
    1 reference
    float minX = -2.5f;
    1 reference
    float maxX = 2.5f;
    1 reference
    float minZ = -2.5f;
    1 reference
    float maxZ = 2.5f;

    // Altura donde es visible, no varia entre instancias
    1 reference
    float altura = 0.085f;

    0 references | Unity Message
    void Start()
    {
        contenedor = GameObject.Find("ContenedorMonedas");
    }

    0 references | Unity Message
    void OnMouseDown()
    {
        // Bucle para instanciar cada moneda
        for (int i = 0; i < cantidadMonedas; i++)
        {
            float posX = Random.Range(minX, maxX);
            float posZ = Random.Range(minZ, maxZ);

            Vector3 posicion = new Vector3(posX, altura, posZ);

            // Instantiate coge cuatro parámetros (objeto a instanciar, lugar donde ponerlo,
            // rotación para asignarle y padre al que va a generar los hijos)
            Instantiate(prefabMoneda, posicion, Quaternion.identity, contenedor.transform);
        }
    }
}
```

InstanciaMonedas está asignado al botón rojo.

BorraInstancias.cs M

Assets > Scripts > E5_Scripts > BorraInstancias.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El segundo destruye todas las monedas que haya
// en la escena en ese momento.
//
//
using UnityEngine;

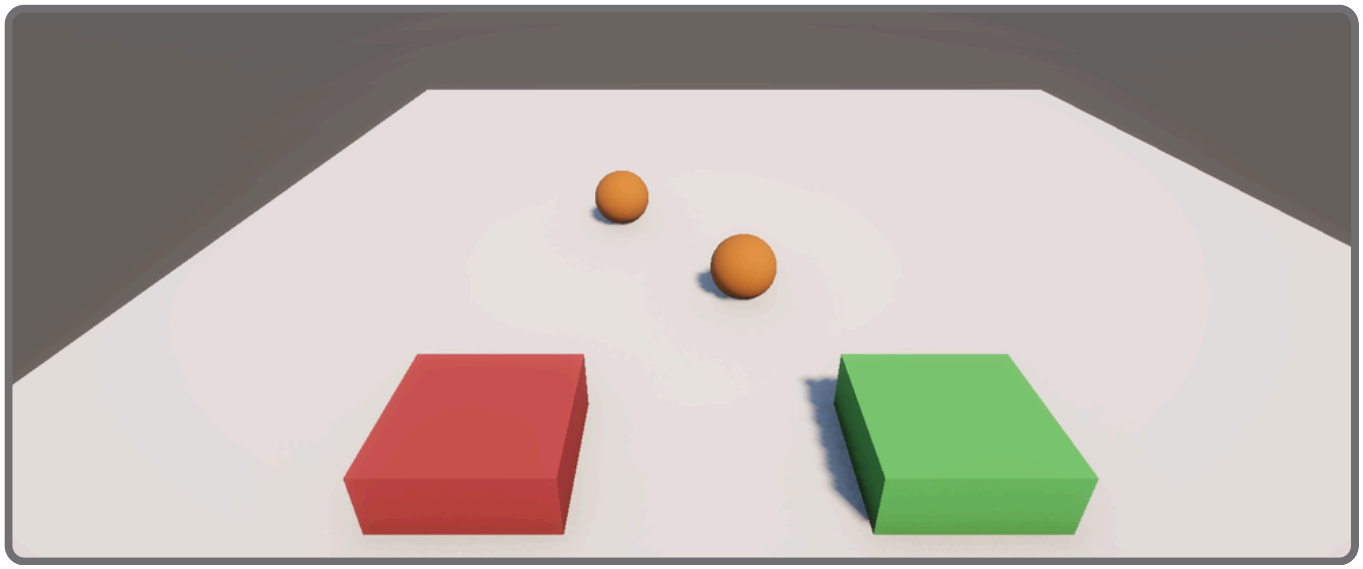
0 references | Unity Script (1 asset reference)
public class BorraInstancias : MonoBehaviour
{
    2 references
    GameObject contenedor;

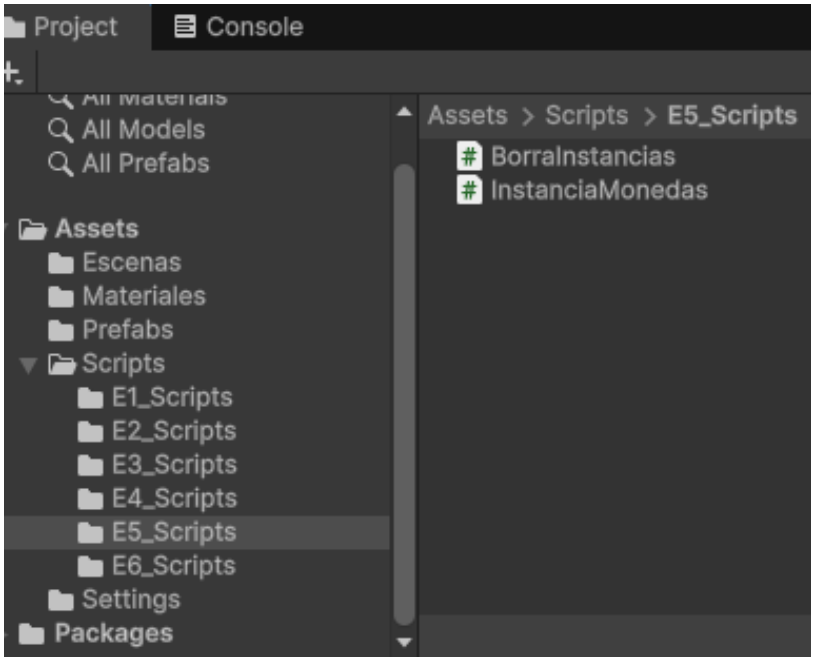
    0 references | Unity Message
    void Start()
    {
        contenedor = GameObject.Find("ContenedorMonedas");
    }

    0 references | Unity Message
    void OnMouseDown()
    {
        foreach (Transform moneda in contenedor.transform)
        {
            // Destroy() borra del hierarchy el objeto (solo en Play Mode)
            Destroy(moneda.gameObject);
        }
    }
}
```

BorraInstancias está asignado al botón verde.

E6_SpawnearLimite:





Para la sexta escena se crean **dos scripts** de MonoBehaviour con las siguientes directrices:

- El primero (verde) genera una esfera con el TAG enemigo en una posición aleatoria dentro del área. Cada pulsación genera una esfera.
- El segundo (rojo) destruye todas las esferas que tengan el TAG enemigo.

Pràctica 2: Ejercicios de Unity básicos

A continuación, se muestra el código de cada script:

SpawnerEnemigosTag.cs M

Assets > Scripts > E6_Scripts > SpawnerEnemigosTag.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El primero (verde) genera una esfera con el TAG enemigo en una posición aleatoria
// dentro del área. Cada pulsación genera una esfera.
//
//
using UnityEngine;

0 references | Unity Script (1 asset reference)
public class SpawnerEnemigosTag : MonoBehaviour
{
    1 reference | Unity Serialized Field = Enemigo
    [SerializeField] GameObject prefabEnemigo;

    2 references
    GameObject contenedor;

    2 references
    int maxEnemigos = 10;

    // Area segura
    1 reference
    float minX = -2.5f;
    1 reference
    float maxX = 2.5f;
    1 reference
    float minZ = -2.5f;
    1 reference
    float maxZ = 2.5f;

    0 references | Unity Message
    void Start()
    {
        contenedor = GameObject.Find("SpawnerEnemigos");
    }

    0 references | Unity Message
    void OnMouseDown()
    {
        // Compruebo la cantidad de enemigos
        int cantidadEnemigos = GameObject.FindGameObjectsWithTag("Enemigo").Length;

        // Si se cumple la condición no hace más de maxEnemigos
        if (cantidadEnemigos >= maxEnemigos)
        {
            Debug.Log("Se ha alcanzado el máximo de "+ maxEnemigos + " enemigos.");
            return;
        }

        // Generar posición aleatoria
        float posX = Random.Range(minX, maxX);
        float posZ = Random.Range(minZ, maxZ);

        // Asigna posicion aleatoria
        Vector3 posicion = new Vector3(posX, 0.17f, posZ);

        // Instancia enemigos dentro de un padre
        Instantiate(prefabEnemigo, posicion, Quaternion.identity, contenedor.transform);
    }
}
```

SpawnerEnemigosTag está asignado al botón verde.

BorraInstanciasTag.cs M

Assets > Scripts > E6_Scripts > BorraInstanciasTag.cs > ...

```
// Programacion y motores de videojuegos: Salva
//
//
// Scripts C# P2 Ejercicios Unity básicos
//
//
// 11/11/2026
// El segundo (rojo) destruye todas las esferas que tengan el TAG enemigo.
//
// Foro de referencia:
// https://discussions.unity.com/t/editor-script-how-to-clear-the-console-output-window/1161
//
using System;
using UnityEngine;
using System.Reflection;
using UnityEditor;

0 references | Unity Script (1 asset reference)
public class BorraInstanciasTag : MonoBehaviour
{
    0 references | Unity Message
    void OnMouseDown()
    {
        // Recorro todos los GamObjects con Flag de "Enemigo"
        foreach (GameObject enemigo in GameObject.FindGameObjectsWithTag("Enemigo"))
        {
            Destroy(enemigo); // Destroy() borra del hierarchy el objeto (solo en Play Mode)
            ClearLog(); // Borra el debug log *Mejora estética*
        }
    }

    // Función sacada de un foro de discusión. Útil para convertir en snippet,
    // usa varias dependencias para funcionar...
    1 reference
    void ClearLog()
    {
        Assembly assembly = Assembly.GetAssembly(typeof(SceneView));

        Type type = assembly.GetType("UnityEditor.LogEntries");
        MethodInfo method = type.GetMethod("Clear");
        method.Invoke(new object(), null);
    }
}
```

SpawnerEnemigosTag está asignado al botón verde.

Conclusión:

Durante esta práctica he aprendido y reforzado conceptos básicos de programación en Unity mediante C#, como el movimiento y la manipulación de GameObjects, el uso de Tags, la gestión de objetos mediante jerarquías de padres e hijos y la interacción con elementos de la escena utilizando OnMouseDown().
También he trabajado con la generación de posiciones aleatorias, la instanciación y destrucción de Prefabs y el control de límites dentro de un área de juego. En general, esta práctica me ha servido para comprender mejor cómo organizar scripts, reutilizar código y aplicar diferentes funcionalidades a los objetos de una escena.